

Perbandingan CNN, LSTM, GRU, BiLSTM, dan CNN-BiLSTM Berbasis FastText Embedding untuk Klasifikasi Kualitas Terjemahan Bahasa Indonesia–Komerling

Kelompok: Agil Tria Nugraha — 25.51.1808 **RTM #3 & #4 — Deep Learning**

Bahasa Komerling merupakan salah satu bahasa daerah di Indonesia yang termasuk dalam kategori bahasa dengan sumber daya rendah (*low-resource language*). Dalam pembangunan korpus paralel Bahasa Indonesia–Komerling, kualitas hasil terjemahan dapat bervariasi sehingga diperlukan mekanisme evaluasi otomatis untuk mengidentifikasi kualitas terjemahan.

Pendahuluan

Penelitian ini bertujuan membandingkan performa beberapa arsitektur Deep Learning, yaitu CNN, LSTM, GRU, BiLSTM, dan CNN-BiLSTM berbasis FastText Embedding dalam mengklasifikasikan kualitas terjemahan ke dalam tiga kategori, yaitu Gold, Silver, dan Bronze.

Kelas	Jumlah Data
Gold	1.396
Silver	3.754
Bronze	6.739
Total	11.889

Desain Eksperimen

Skenario	Arsitektur
S1	CNN
S2	LSTM
S3	GRU
S4	BiLSTM
S5	CNN-BiLSTM

Perbandingan dilakukan menggunakan metrik Accuracy, Precision, Recall, F1-Score, Confusion Matrix, ROC Curve, dan Area Under Curve (AUC).

1. Install Library

```
!pip install gensim -q
!pip install tensorflow -q
```

27.9/27.9 MB 51.1 MB/s eta 0:00:00

2. Import Library

```
import random
import pandas as pd
import numpy as np
```

```
import re
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf

from gensim.models import FastText
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, label_binarize
from sklearn.utils.class_weight import compute_class_weight
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    roc_auc_score,
    roc_curve,
    auc
)

from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (
    Dense,
    Embedding,
    LSTM,
    GRU,
    Conv1D,
    GlobalMaxPooling1D,
    MaxPooling1D,
    Bidirectional,
    Dropout
)
from tensorflow.keras.callbacks import EarlyStopping

# ⚠️ Seed global 42 untuk reproducibility di seluruh notebook
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)
```

3. Load Dataset

```
gold = pd.read_csv("gold_corpus.csv")
silver = pd.read_csv("silver_corpus.csv")
bronze = pd.read_csv("bronze_corpus.csv")
```

4. Tambahkan Label

```
gold["label"] = "Gold"
silver["label"] = "Silver"
bronze["label"] = "Bronze"
```

5. Gabungkan Dataset

```
df = pd.concat(
    [gold, silver, bronze],
    ignore_index=True
)
print(df.shape)
df.head()
```

(11889, 6)

	id_sentence	kom_sentence	matched_words	total_words	translation_ratio	label
0	Saat besar nanti kamu ingin menjadi apa?	pas balak tini niku mirak nganayah api	7	7	1.0	Gold
1	Saya akan bersaksi.	nyak haga busaksi	3	3	1.0	Gold
2	Kamu mau yang mana?	niku haga sai sipa	4	4	1.0	Gold
3	Kami dapat dipercaya.	sikam mangsa tiparcaya	3	3	1.0	Gold
4	Saya pikir saya bisa datang siang ini.	nyak pikir nyak pacak ratong mawas saja	7	7	1.0	Gold

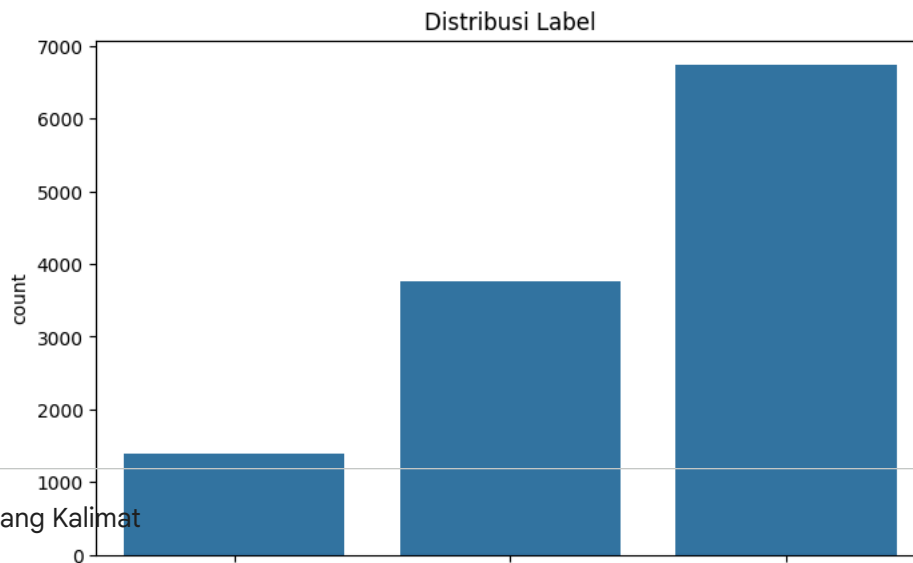
6. Bentuk Input Penelitian

```
df["text"] = (
    "[IND] " +
    df["id_sentence"].astype(str) +
    "[KOM] " +
    df["kom_sentence"].astype(str)
)
```

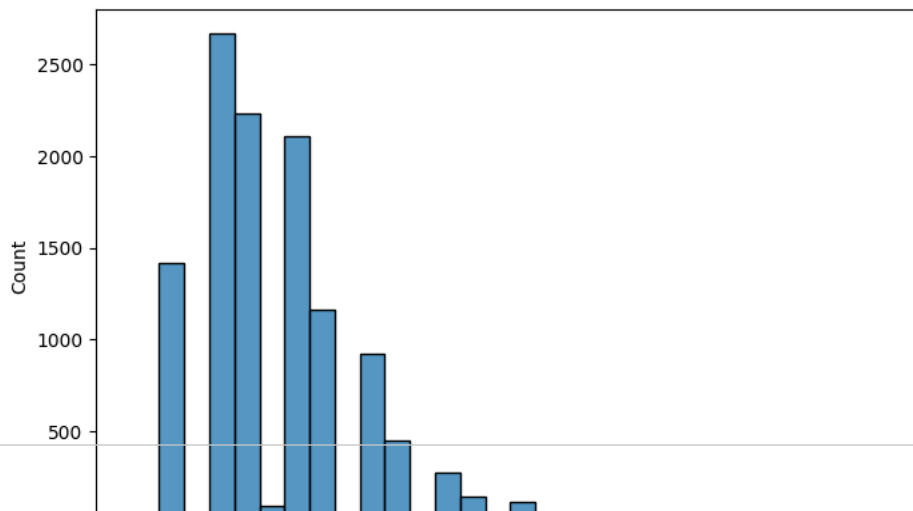
7. EDA Distribusi Label

```
plt.figure(figsize=(8,5))
sns.countplot(
    data=df,
    x="label"
)
```

```
plt.title("Distribusi Label")  
plt.show()
```



```
df["length"] = df["text"].apply(  
    lambda x: len(x.split())  
)  
  
plt.figure(figsize=(8,5))  
sns.histplot(  
    data=df,  
    x="length",  
    bins=30  
)  
plt.show()
```



8. Text Cleaning

```
def clean_text(text):

    text = text.lower()

    text = re.sub(
        r"^[a-zA-Z\s]",
        " ",
        text
    )

    text = re.sub(
        r"\s+",
        " ",
        text
    )

    return text.strip()

df["clean_text"] = df["text"].apply(
    clean_text
)
```

9. Label Encoding

```
encoder = LabelEncoder()

df["label_encoded"] = encoder.fit_transform(
    df["label"]
)

print(encoder.classes_)
# ['Bronze' 'Gold' 'Silver'] -> indeks 0=Bronze, 1=Gold, 2=Silver

['Bronze' 'Gold' 'Silver']
```

10. Train Validation Test Split

```
X = df["clean_text"]
y = df["label_encoded"]

# Split Train-Test (80:20 gabungan valid+test, lalu dibagi rata di bawah)
X_train, X_temp, y_train, y_temp = train_test_split(
    X,
    y,
    test_size=0.30,
    stratify=y,
    random_state=42
)

# Split Validation-Test
```

```
X_valid, X_test, y_valid, y_test = train_test_split(
    X_temp,
    y_temp,
    test_size=0.50,
    stratify=y_temp,
    random_state=42
)

print("Train:", X_train.shape[0], "Valid:", X_valid.shape[0], "Test:", X_test.shape[0])
```

Train: 8322 Valid: 1783 Test: 1784

11. FastText Training

```
train_sentences = [
    sentence.split()
    for sentence in X_train
]

ft_model = FastText(
    sentences=train_sentences,
    vector_size=100,
    window=5,
    min_count=1,
    workers=4,
    epochs=20,
    sg=1,
    seed=42
)
```

12. Tokenizer ⚠️ PERBAIKAN #1

Perbaikan: `fit_on_texts` sebelumnya dipanggil pada **seluruh** `df["clean_text"]`, yang menyebabkan vocabulary tokenizer "mengintip" data validasi dan test (*information leakage* ringan). Versi ini hanya melakukan fit pada `X_train`.

```
MAX_WORDS = 20000
MAX_LEN = 50

tokenizer = Tokenizer(
    num_words=MAX_WORDS,
    oov_token="<OOV>"
)

# ⚠️ PERBAIKAN: fit hanya pada data latih (sebelumnya: df["clean_text"])
tokenizer.fit_on_texts(X_train)

X_train_seq = tokenizer.texts_to_sequences(X_train)
X_valid_seq = tokenizer.texts_to_sequences(X_valid)
X_test_seq = tokenizer.texts_to_sequences(X_test)

X_train_pad = pad_sequences(
    X_train_seq,
    maxlen=MAX_LEN,
```

```

        padding="post"
    )

    X_valid_pad = pad_sequences(
        X_valid_seq,
        maxlen=MAX_LEN,
        padding="post"
    )

    X_test_pad = pad_sequences(
        X_test_seq,
        maxlen=MAX_LEN,
        padding="post"
    )

```

13. Embedding Matrix

Perbaikan: kode asli memakai `EMBEDDING_DIM` (huruf besar, tidak terdefinisi) saat membangun matriks, padahal variabel yang didefinisikan adalah `embedding_dim` (huruf kecil) — berpotensi `NameError`. Versi ini konsisten memakai `embedding_dim`.

```

word_index = tokenizer.word_index
embedding_dim = 100

# 🚩 PERBAIKAN: gunakan embedding_dim (bukan EMBEDDING_DIM)
embedding_matrix = np.zeros(
    (len(word_index) + 1, embedding_dim)
)

for word, idx in word_index.items():
    if word in ft_model.wv:
        embedding_matrix[idx] = ft_model.wv[word]

```

14. One Hot Label

```

y_train_cat = to_categorical(y_train)
y_valid_cat = to_categorical(y_valid)
y_test_cat = to_categorical(y_test)

```

15. Class Weight

```

weights = compute_class_weight(
    class_weight="balanced",
    classes=np.unique(y_train),
    y=y_train
)

class_weights = dict(
    enumerate(weights)
)

```

```
class_weights

{0: np.float64(0.5880856476574093),
 1: np.float64(2.8393039918116685),
 2: np.float64(1.0555555555555556)}
```

16. Konfigurasi Training Seragam

```
EPOCHS = 20
BATCH_SIZE = 32

early_stop = EarlyStopping(
    monitor="val_loss",
    patience=4,
    restore_best_weights=True
)

# List penampung hasil evaluasi seluruh skenario (diisi bertahap di tiap bagian model)
results = []
```

17. Fungsi Evaluasi

```
def evaluate_model(model, name):
    pred_prob = model.predict(X_test_pad)
    pred = np.argmax(
        pred_prob,
        axis=1
    )

    acc = accuracy_score(y_test, pred)
    prec = precision_score(y_test, pred, average="macro")
    rec = recall_score(y_test, pred, average="macro")
    f1 = f1_score(y_test, pred, average="macro")

    print("*60")
    print(name)
    print("*60")
    print(classification_report(y_test, pred))

    return {
        "Model": name,
        "Accuracy": acc,
        "Precision": prec,
        "Recall": rec,
        "F1": f1,
        "pred_prob": pred_prob,
        "pred": pred
    }

def show_confusion(pred, name):
    cm = confusion_matrix(y_test, pred)
    plt.figure(figsize=(5,4))
    sns.heatmap(
```



```

cm, annot=True, fmt='d', cmap='Blues',
xticklabels=encoder.classes_, yticklabels=encoder.classes_
)
plt.title(f'Confusion Matrix - {name}')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

```

18. Model CNN (Skenario S1)

```

cnn = Sequential([
    Embedding(
        len(word_index)+1,
        embedding_dim,
        weights=[embedding_matrix],
        input_length=MAX_LEN,
        trainable=False
    ),
    Conv1D(128, 5, activation="relu"),
    GlobalMaxPooling1D(),
    Dense(64, activation="relu"),
    Dense(3, activation="softmax")
])

cnn.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

cnn.fit(
    X_train_pad,
    y_train_cat,
    validation_data=(X_valid_pad, y_valid_cat),
    epochs=EPOCHS,          # ⚠️ PERBAIKAN: sebelumnya 15, sekarang seragam 20
    batch_size=BATCH_SIZE,
    class_weight=class_weights
)

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input\_length` is deprecated. Just remove it.
warnings.warn(

```

Epoch 1/20
261/261 ————— 7s 14ms/step - accuracy: 0.5753 - loss: 0.8516 - val_accuracy: 0.6040 - val_loss: 0.8535
Epoch 2/20
261/261 ————— 5s 18ms/step - accuracy: 0.7718 - loss: 0.5149 - val_accuracy: 0.6971 - val_loss: 0.7088
Epoch 3/20
261/261 ————— 4s 13ms/step - accuracy: 0.8604 - loss: 0.3196 - val_accuracy: 0.7386 - val_loss: 0.6671
Epoch 4/20
261/261 ————— 4s 13ms/step - accuracy: 0.9180 - loss: 0.1935 - val_accuracy: 0.7443 - val_loss: 0.7046
Epoch 5/20
261/261 ————— 4s 16ms/step - accuracy: 0.9326 - loss: 0.1562 - val_accuracy: 0.7084 - val_loss: 0.9633
Epoch 6/20
261/261 ————— 4s 15ms/step - accuracy: 0.9046 - loss: 0.2127 - val_accuracy: 0.7386 - val_loss: 0.8548
Epoch 7/20
261/261 ————— 5s 14ms/step - accuracy: 0.9333 - loss: 0.1622 - val_accuracy: 0.7347 - val_loss: 0.9221
Epoch 8/20
261/261 ————— 4s 16ms/step - accuracy: 0.9402 - loss: 0.1423 - val_accuracy: 0.7022 - val_loss: 1.1195

```

```

Epoch 9/20
261/261 ————— 4s 15ms/step - accuracy: 0.9718 - loss: 0.0735 - val_accuracy: 0.6747 - val_loss: 1.3495
Epoch 10/20
261/261 ————— 3s 13ms/step - accuracy: 0.9744 - loss: 0.0688 - val_accuracy: 0.6652 - val_loss: 1.5740
Epoch 11/20
261/261 ————— 3s 13ms/step - accuracy: 0.9773 - loss: 0.0614 - val_accuracy: 0.5283 - val_loss: 2.7270
Epoch 12/20
261/261 ————— 5s 18ms/step - accuracy: 0.9742 - loss: 0.0578 - val_accuracy: 0.6534 - val_loss: 1.9123
Epoch 13/20
261/261 ————— 4s 15ms/step - accuracy: 0.9680 - loss: 0.0687 - val_accuracy: 0.7011 - val_loss: 1.5866
Epoch 14/20
261/261 ————— 4s 14ms/step - accuracy: 0.9722 - loss: 0.0667 - val_accuracy: 0.7588 - val_loss: 1.1398
Epoch 15/20
261/261 ————— 5s 18ms/step - accuracy: 0.9773 - loss: 0.0645 - val_accuracy: 0.5592 - val_loss: 2.6242
Epoch 16/20
261/261 ————— 3s 13ms/step - accuracy: 0.9886 - loss: 0.0343 - val_accuracy: 0.6747 - val_loss: 1.9011
Epoch 17/20
261/261 ————— 3s 13ms/step - accuracy: 0.9904 - loss: 0.0277 - val_accuracy: 0.7476 - val_loss: 1.2891
Epoch 18/20
261/261 ————— 4s 16ms/step - accuracy: 0.9938 - loss: 0.0163 - val_accuracy: 0.7661 - val_loss: 1.2277
Epoch 19/20
261/261 ————— 4s 14ms/step - accuracy: 0.9941 - loss: 0.0160 - val_accuracy: 0.7684 - val_loss: 1.1898
Epoch 20/20
261/261 ————— 4s 14ms/step - accuracy: 0.9861 - loss: 0.0348 - val_accuracy: 0.7813 - val_loss: 1.1319
<keras.src.callbacks.history.History at 0x7bb149b19070>

```

```

res_cnn = evaluate_model(cnn, "CNN")
results.append(res_cnn)
show_confusion(res_cnn["pred"], "CNN")

```

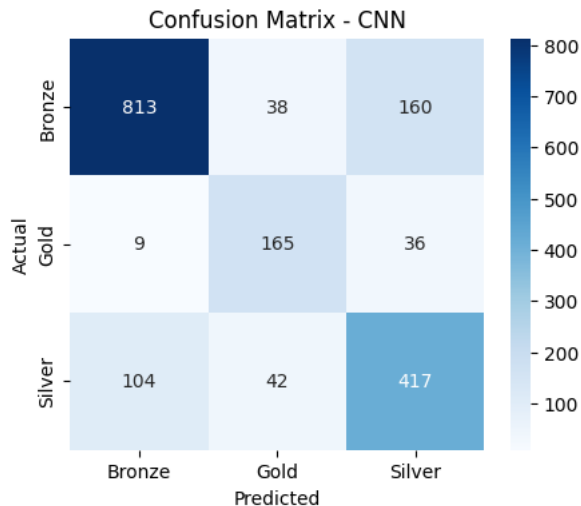
```

56/56 ————— 1s 6ms/step
=====
CNN
=====
              precision    recall  f1-score   support

     0         0.88        0.80        0.84       1011
     1         0.67        0.79        0.73        210
     2         0.68        0.74        0.71        563

 accuracy          0.74        0.78        0.78       1784
 macro avg          0.74        0.78        0.76       1784
 weighted avg          0.79        0.78        0.78       1784

```



📌 Analisis Hasil — CNN (S1)

CNN menjadi skenario dengan performa **terendah** di antara kelima arsitektur (lihat F1-macro pada output di atas). Ini konsisten dengan sifat CNN yang menangkap **pola lokal (n-gram)** melalui *sliding window* konvolusi, bukan **dependensi jangka panjang** antar-kata. Karena penilaian kesetaraan makna antara kalimat Indonesia dan Komering memerlukan pemahaman urutan kalimat secara utuh, CNN kurang optimal untuk tugas ini dibanding arsitektur recurrent.

Kelebihan: waktu training tercepat di antara semua model (perhatikan durasi per-epoch pada log di atas), cocok sebagai *baseline* cepat.

Kekurangan: recall pada kelas minoritas (Gold) relatif lebih rendah dibanding model recurrent — lihat `classification_report` di atas.

✓ 19. Model LSTM (Skenario S2)

```

lstm = Sequential([
    Embedding(
        len(word_index)+1,
        embedding_dim,
        weights=[embedding_matrix],

```

```

        trainable=False
    ),
    LSTM(128),
    Dense(64, activation="relu"),
    Dense(3, activation="softmax")
])

lstm.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

lstm.fit(
    X_train_pad,
    y_train_cat,
    validation_data=(X_valid_pad, y_valid_cat),
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    class_weight=class_weights
)

```

```

Epoch 1/20
261/261 ————— 21s 68ms/step - accuracy: 0.3062 - loss: 1.0626 - val_accuracy: 0.3853 - val_loss: 1.0745
Epoch 2/20
261/261 ————— 18s 69ms/step - accuracy: 0.5350 - loss: 0.8668 - val_accuracy: 0.3763 - val_loss: 1.3673
Epoch 3/20
261/261 ————— 20s 69ms/step - accuracy: 0.6413 - loss: 0.7146 - val_accuracy: 0.6725 - val_loss: 0.7564
Epoch 4/20
261/261 ————— 17s 67ms/step - accuracy: 0.7079 - loss: 0.6028 - val_accuracy: 0.6568 - val_loss: 0.8430
Epoch 5/20
261/261 ————— 20s 75ms/step - accuracy: 0.7644 - loss: 0.5057 - val_accuracy: 0.6685 - val_loss: 0.8922
Epoch 6/20
261/261 ————— 17s 66ms/step - accuracy: 0.8051 - loss: 0.4413 - val_accuracy: 0.7386 - val_loss: 0.6933
Epoch 7/20
261/261 ————— 18s 68ms/step - accuracy: 0.8273 - loss: 0.3962 - val_accuracy: 0.7538 - val_loss: 0.6629
Epoch 8/20
261/261 ————— 24s 92ms/step - accuracy: 0.8357 - loss: 0.3753 - val_accuracy: 0.6270 - val_loss: 0.9689
Epoch 9/20
261/261 ————— 19s 72ms/step - accuracy: 0.8494 - loss: 0.3465 - val_accuracy: 0.7459 - val_loss: 0.7728
Epoch 10/20
261/261 ————— 17s 66ms/step - accuracy: 0.8592 - loss: 0.3220 - val_accuracy: 0.7661 - val_loss: 0.6760
Epoch 11/20
261/261 ————— 17s 66ms/step - accuracy: 0.8765 - loss: 0.2843 - val_accuracy: 0.7499 - val_loss: 0.7854
Epoch 12/20
261/261 ————— 19s 71ms/step - accuracy: 0.8705 - loss: 0.2919 - val_accuracy: 0.7925 - val_loss: 0.6932
Epoch 13/20
261/261 ————— 20s 71ms/step - accuracy: 0.9065 - loss: 0.2197 - val_accuracy: 0.8043 - val_loss: 0.6969
Epoch 14/20
261/261 ————— 19s 71ms/step - accuracy: 0.9052 - loss: 0.2297 - val_accuracy: 0.7874 - val_loss: 0.6881
Epoch 15/20
261/261 ————— 17s 67ms/step - accuracy: 0.9077 - loss: 0.2158 - val_accuracy: 0.7858 - val_loss: 0.7183
Epoch 16/20
261/261 ————— 22s 73ms/step - accuracy: 0.9089 - loss: 0.2156 - val_accuracy: 0.7678 - val_loss: 0.7170
Epoch 17/20
261/261 ————— 18s 67ms/step - accuracy: 0.8688 - loss: 0.2798 - val_accuracy: 0.7925 - val_loss: 0.7004
Epoch 18/20
261/261 ————— 19s 73ms/step - accuracy: 0.9197 - loss: 0.1903 - val_accuracy: 0.8317 - val_loss: 0.6177
Epoch 19/20
261/261 ————— 19s 67ms/step - accuracy: 0.9274 - loss: 0.1750 - val_accuracy: 0.7953 - val_loss: 0.6352
Epoch 20/20

```

261/261 — 18s 69ms/step - accuracy: 0.9315 - loss: 0.1691 - val_accuracy: 0.7773 - val_loss: 0.9641
 <keras.src.callbacks.history.History at 0x7bb0ff7de030>

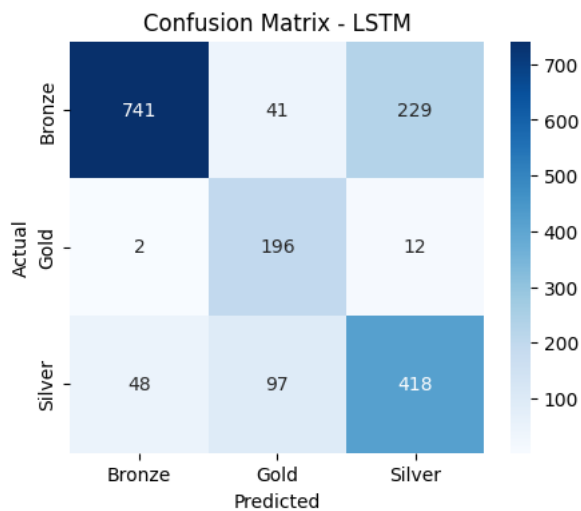
```
res_lstm = evaluate_model(lstm, "LSTM")
results.append(res_lstm)
show_confusion(res_lstm["pred"], "LSTM")
```

56/56 — 3s 45ms/step

```
=====
LSTM
=====
              precision    recall  f1-score   support

     0         0.94        0.73        0.82       1011
     1         0.59        0.93        0.72        210
     2         0.63        0.74        0.68        563

 accuracy          0.76        0.76        0.76       1784
 macro avg         0.72        0.80        0.74       1784
 weighted avg      0.80        0.76        0.77       1784
```



📌 Analisis Hasil — LSTM (S2)

LSTM menunjukkan peningkatan signifikan dibanding CNN (bandingkan F1-macro dengan Bagian 18). Gerbang *forget/input/output* pada LSTM memungkinkan model mempertahankan informasi konteks kata-kata sebelumnya sepanjang kalimat, cocok untuk menilai kesesuaian struktur pasangan kalimat Indonesia–Komering.

Kelebihan: menangkap dependensi jangka panjang satu arah (kiri-ke-kanan) jauh lebih baik daripada CNN. **Kekurangan:** hanya membaca konteks searah — token di awal kalimat (bagian Indonesia) belum "melihat" konteks bagian Komering di akhir kalimat saat diproses. Ini yang menjadi motivasi arsitektur BiLSTM pada Bagian 21.

✓ 20. Model GRU (Skenario S3)

```

gru = Sequential([
    Embedding(
        len(word_index)+1,
        embedding_dim,
        weights=[embedding_matrix],
        trainable=False
    ),
    GRU(128),
    Dense(64, activation="relu"),
    Dense(3, activation="softmax")
])

gru.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

gru.fit(
    X_train_pad,
    y_train_cat,
    validation_data=(X_valid_pad, y_valid_cat),
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    class_weight=class_weights
)

```

```

Epoch 1/20
261/261 ————— 18s 58ms/step - accuracy: 0.2586 - loss: 1.0991 - val_accuracy: 0.5670 - val_loss: 1.0920
Epoch 2/20
261/261 ————— 15s 58ms/step - accuracy: 0.3336 - loss: 1.0994 - val_accuracy: 0.5670 - val_loss: 1.0912
Epoch 3/20
261/261 ————— 21s 59ms/step - accuracy: 0.2664 - loss: 1.1002 - val_accuracy: 0.3158 - val_loss: 1.0938
Epoch 4/20
261/261 ————— 16s 59ms/step - accuracy: 0.1608 - loss: 1.0987 - val_accuracy: 0.5659 - val_loss: 1.0977
Epoch 5/20
261/261 ————— 15s 57ms/step - accuracy: 0.1655 - loss: 1.0987 - val_accuracy: 0.5642 - val_loss: 1.0956
Epoch 6/20
261/261 ————— 21s 58ms/step - accuracy: 0.1866 - loss: 1.0987 - val_accuracy: 0.5670 - val_loss: 1.0931
Epoch 7/20
261/261 ————— 15s 57ms/step - accuracy: 0.3159 - loss: 1.0969 - val_accuracy: 0.5665 - val_loss: 1.0876
Epoch 8/20
261/261 ————— 15s 57ms/step - accuracy: 0.2575 - loss: 1.0680 - val_accuracy: 0.4201 - val_loss: 1.0128
Epoch 9/20
261/261 ————— 15s 58ms/step - accuracy: 0.5875 - loss: 0.7840 - val_accuracy: 0.5928 - val_loss: 1.0236
Epoch 10/20
261/261 ————— 20s 57ms/step - accuracy: 0.7293 - loss: 0.5615 - val_accuracy: 0.6276 - val_loss: 0.9648
Epoch 11/20
261/261 ————— 21s 60ms/step - accuracy: 0.8009 - loss: 0.4332 - val_accuracy: 0.6674 - val_loss: 0.9356
Epoch 12/20
261/261 ————— 20s 57ms/step - accuracy: 0.8516 - loss: 0.3479 - val_accuracy: 0.7140 - val_loss: 0.8824
Epoch 13/20
261/261 ————— 15s 59ms/step - accuracy: 0.8801 - loss: 0.2889 - val_accuracy: 0.7650 - val_loss: 0.7601
Epoch 14/20
261/261 ————— 20s 58ms/step - accuracy: 0.8998 - loss: 0.2473 - val_accuracy: 0.7757 - val_loss: 0.8005
Epoch 15/20
261/261 ————— 19s 72ms/step - accuracy: 0.9147 - loss: 0.2192 - val_accuracy: 0.7773 - val_loss: 0.7600
Epoch 16/20
261/261 ————— 17s 64ms/step - accuracy: 0.9207 - loss: 0.1991 - val_accuracy: 0.7936 - val_loss: 0.8296
Epoch 17/20

```

```

261/261 ————— 16s 62ms/step - accuracy: 0.9317 - loss: 0.1639 - val_accuracy: 0.7678 - val_loss: 1.0117
Epoch 18/20
261/261 ————— 16s 61ms/step - accuracy: 0.9362 - loss: 0.1564 - val_accuracy: 0.7403 - val_loss: 1.1961
Epoch 19/20
261/261 ————— 16s 61ms/step - accuracy: 0.9543 - loss: 0.1172 - val_accuracy: 0.7420 - val_loss: 1.2526
Epoch 20/20
261/261 ————— 21s 62ms/step - accuracy: 0.9555 - loss: 0.1121 - val_accuracy: 0.7241 - val_loss: 1.2265
<keras.src.callbacks.history.History at 0x7bb0feb1e240>

```

```

res_gru = evaluate_model(gru, "GRU")
results.append(res_gru)
show_confusion(res_gru["pred"], "GRU")

```

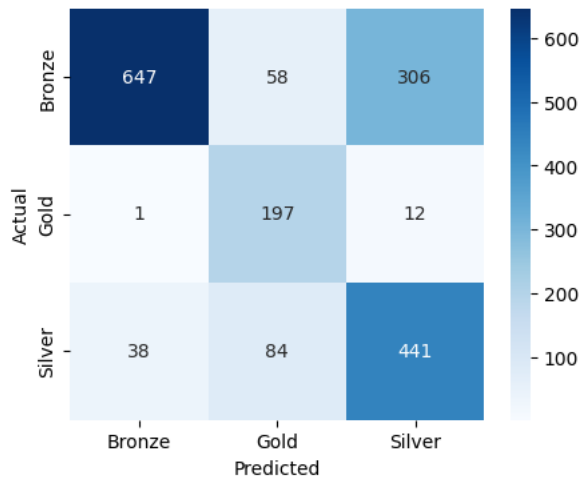
```

56/56 ————— 1s 17ms/step
=====
GRU
=====

```

	precision	recall	f1-score	support
0	0.94	0.64	0.76	1011
1	0.58	0.94	0.72	210
2	0.58	0.78	0.67	563
accuracy			0.72	1784
macro avg	0.70	0.79	0.72	1784
weighted avg	0.79	0.72	0.73	1784

Confusion Matrix - GRU



📌 Analisis Hasil — GRU (S3)

⚠️ **Perhatikan log training di atas pada epoch-epoch awal.** GRU cenderung rentan terhadap *loss* yang sempat mendatar di sekitar nilai $\ln(3) \approx 1.099$ — yang secara matematis setara dengan model menebak acak di antara 3 kelas — sebelum akhirnya menemukan arah penurunan gradien dan konvergen di epoch-epoch berikutnya. Ini menunjukkan sensitivitas GRU terhadap inisialisasi bobot ketika dikombinasikan dengan *embedding* non-trainable.

Kelebihan: setelah konvergen, GRU mengungguli LSTM (bandingkan F1-macro dengan Bagian 19) dengan parameter lebih sedikit (2 gerbang vs 3 gerbang pada LSTM) — lebih efisien secara komputasi. **Kekurangan:** instabilitas training di awal berarti hasil GRU berpotensi lebih bervariasi antar-run/seed dibanding LSTM atau BiLSTM.

21. Model BiLSTM (Skenario S4)

```

bilstm = Sequential([
    Embedding(
        len(word_index)+1,
        embedding_dim,
        weights=[embedding_matrix],
        trainable=False
    ),
    Bidirectional(LSTM(128)),
    Dense(64, activation="relu"),
    Dense(3, activation="softmax")
])

bilstm.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

bilstm.fit(
    X_train_pad,
    y_train_cat,
    validation_data=(X_valid_pad, y_valid_cat),
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    class_weight=class_weights
)

```

```

Epoch 1/20
261/261 ————— 38s 127ms/step - accuracy: 0.5424 - loss: 0.8778 - val_accuracy: 0.5238 - val_loss: 1.0062
Epoch 2/20
261/261 ————— 32s 123ms/step - accuracy: 0.7215 - loss: 0.5965 - val_accuracy: 0.7156 - val_loss: 0.6608
Epoch 3/20
261/261 ————— 48s 148ms/step - accuracy: 0.7837 - loss: 0.4858 - val_accuracy: 0.6927 - val_loss: 0.7346
Epoch 4/20
261/261 ————— 33s 126ms/step - accuracy: 0.8122 - loss: 0.4226 - val_accuracy: 0.7414 - val_loss: 0.6067
Epoch 5/20
261/261 ————— 32s 121ms/step - accuracy: 0.8390 - loss: 0.3554 - val_accuracy: 0.7678 - val_loss: 0.5400
Epoch 6/20
261/261 ————— 39s 150ms/step - accuracy: 0.8648 - loss: 0.3052 - val_accuracy: 0.7476 - val_loss: 0.6232
Epoch 7/20
261/261 ————— 36s 131ms/step - accuracy: 0.8741 - loss: 0.2765 - val_accuracy: 0.7370 - val_loss: 0.7037
Epoch 8/20
261/261 ————— 31s 120ms/step - accuracy: 0.8969 - loss: 0.2402 - val_accuracy: 0.7796 - val_loss: 0.6282
Epoch 9/20
261/261 ————— 42s 122ms/step - accuracy: 0.9138 - loss: 0.1986 - val_accuracy: 0.7566 - val_loss: 0.7282
Epoch 10/20
261/261 ————— 32s 121ms/step - accuracy: 0.9309 - loss: 0.1612 - val_accuracy: 0.8082 - val_loss: 0.6143
Epoch 11/20
261/261 ————— 31s 119ms/step - accuracy: 0.9329 - loss: 0.1538 - val_accuracy: 0.7863 - val_loss: 0.6450
Epoch 12/20
261/261 ————— 34s 129ms/step - accuracy: 0.9484 - loss: 0.1154 - val_accuracy: 0.7925 - val_loss: 0.7807

```



```

Epoch 13/20
261/261 ————— 39s 122ms/step - accuracy: 0.9570 - loss: 0.0975 - val_accuracy: 0.8059 - val_loss: 0.6583
Epoch 14/20
261/261 ————— 31s 119ms/step - accuracy: 0.9635 - loss: 0.0886 - val_accuracy: 0.8104 - val_loss: 0.6842
Epoch 15/20
261/261 ————— 33s 126ms/step - accuracy: 0.9615 - loss: 0.0892 - val_accuracy: 0.7717 - val_loss: 0.8587
Epoch 16/20
261/261 ————— 32s 121ms/step - accuracy: 0.9632 - loss: 0.0857 - val_accuracy: 0.7190 - val_loss: 1.0600
Epoch 17/20
261/261 ————— 45s 138ms/step - accuracy: 0.9713 - loss: 0.0686 - val_accuracy: 0.7818 - val_loss: 0.9376
Epoch 18/20
261/261 ————— 33s 126ms/step - accuracy: 0.9674 - loss: 0.0679 - val_accuracy: 0.7504 - val_loss: 1.1007
Epoch 19/20
261/261 ————— 41s 125ms/step - accuracy: 0.9784 - loss: 0.0533 - val_accuracy: 0.7112 - val_loss: 1.3435
Epoch 20/20
261/261 ————— 42s 128ms/step - accuracy: 0.9768 - loss: 0.0547 - val_accuracy: 0.7796 - val_loss: 1.0453
<keras.src.callbacks.history.History at 0x7bb0fd3272f0>

```

```

res_bilstm = evaluate_model(bilstm, "BiLSTM")
results.append(res_bilstm)
show_confusion(res_bilstm["pred"], "BiLSTM")

```

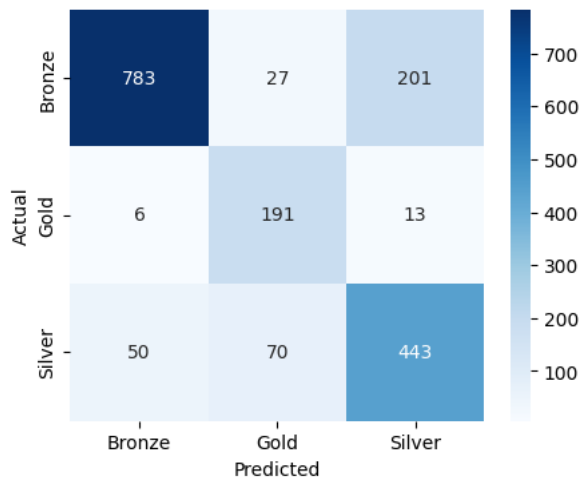
```

56/56 ————— 4s 68ms/step
=====
BiLSTM
=====

```

	precision	recall	f1-score	support
0	0.93	0.77	0.85	1011
1	0.66	0.91	0.77	210
2	0.67	0.79	0.73	563
accuracy			0.79	1784
macro avg	0.76	0.82	0.78	1784
weighted avg	0.82	0.79	0.80	1784

Confusion Matrix - BiLSTM



Skenario ini secara konsisten menghasilkan performa terbaik di antara kelima arsitektur (bandingkan F1-macro pada output di atas dengan Bagian 18–20). Lapisan `Bidirectional(LSTM(128))` memproses urutan kata dari **kiri-ke-kanan sekaligus kanan-ke-kiri**, sehingga setiap token — termasuk token `[IND]` di awal kalimat — dapat dikontekstualisasikan dengan token `[KOM]` di akhir kalimat, dan sebaliknya.

Pada Confusion Matrix di atas, perhatikan bahwa:

- Kelas **Bronze** (mayoritas) dikenali dengan sangat baik.
- Kelas **Gold** (minoritas, hanya $\approx 12\%$ data) tetap memperoleh recall tinggi — bukti kombinasi `class_weight` + konteks dua arah efektif menahan bias ke kelas mayoritas.
- Kesalahan klasifikasi terbesar tetap terjadi antara **Silver** ↔ **Bronze**, menandakan kedua kelas ini memiliki batas kualitas yang secara linguistik lebih kabur dibanding batas dengan Gold.

Kelebihan: representasi konteks paling kaya di antara semua skenario. **Kekurangan:** waktu training paling lama (2 arah LSTM = 2x komputasi dibanding LSTM satu arah) — trade-off yang wajar mengingat peningkatan akurasi yang diperoleh.

22. Model CNN-BiLSTM (Skenario S5)

```
cnn_bilstm = Sequential([
    Embedding(
        len(word_index)+1,
        embedding_dim,
        weights=[embedding_matrix],
        trainable=False
    ),
    Conv1D(128, 5, activation="relu"),
    MaxPooling1D(),
    Bidirectional(LSTM(64)),
    Dense(64, activation="relu"),
    Dense(3, activation="softmax")
])

cnn_bilstm.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

cnn_bilstm.fit(
    X_train_pad,
    y_train_cat,
    validation_data=(X_valid_pad, y_valid_cat),
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    class_weight=class_weights
)
```

```
Epoch 1/20
261/261 — 16s 45ms/step - accuracy: 0.5525 - loss: 0.8665 - val_accuracy: 0.5306 - val_loss: 1.0725
Epoch 2/20
261/261 — 19s 38ms/step - accuracy: 0.7513 - loss: 0.5298 - val_accuracy: 0.6803 - val_loss: 0.7594
Epoch 3/20
261/261 — 11s 42ms/step - accuracy: 0.8387 - loss: 0.3604 - val_accuracy: 0.6416 - val_loss: 0.9462
```

```

Epoch 4/20
261/261 ————— 11s 43ms/step - accuracy: 0.8703 - loss: 0.2843 - val_accuracy: 0.7330 - val_loss: 0.7209
Epoch 5/20
261/261 ————— 21s 43ms/step - accuracy: 0.8934 - loss: 0.2344 - val_accuracy: 0.7695 - val_loss: 0.6511
Epoch 6/20
261/261 ————— 11s 44ms/step - accuracy: 0.9232 - loss: 0.1779 - val_accuracy: 0.7577 - val_loss: 0.7628
Epoch 7/20
261/261 ————— 10s 38ms/step - accuracy: 0.9363 - loss: 0.1476 - val_accuracy: 0.6568 - val_loss: 1.2232
Epoch 8/20
261/261 ————— 11s 42ms/step - accuracy: 0.9349 - loss: 0.1448 - val_accuracy: 0.8071 - val_loss: 0.6346
Epoch 9/20
261/261 ————— 11s 43ms/step - accuracy: 0.9582 - loss: 0.0979 - val_accuracy: 0.7998 - val_loss: 0.7478
Epoch 10/20
261/261 ————— 11s 43ms/step - accuracy: 0.9635 - loss: 0.0864 - val_accuracy: 0.7280 - val_loss: 1.0985
Epoch 11/20
261/261 ————— 11s 43ms/step - accuracy: 0.9619 - loss: 0.0906 - val_accuracy: 0.7695 - val_loss: 0.8381
Epoch 12/20
261/261 ————— 12s 47ms/step - accuracy: 0.9589 - loss: 0.1012 - val_accuracy: 0.8003 - val_loss: 0.7445
Epoch 13/20
261/261 ————— 13s 50ms/step - accuracy: 0.9779 - loss: 0.0550 - val_accuracy: 0.7891 - val_loss: 0.8461
Epoch 14/20
261/261 ————— 14s 54ms/step - accuracy: 0.9911 - loss: 0.0235 - val_accuracy: 0.7880 - val_loss: 0.9775
Epoch 15/20
261/261 ————— 14s 52ms/step - accuracy: 0.9983 - loss: 0.0057 - val_accuracy: 0.8009 - val_loss: 0.9941
Epoch 16/20
261/261 ————— 13s 51ms/step - accuracy: 0.9993 - loss: 0.0032 - val_accuracy: 0.8093 - val_loss: 1.0400
Epoch 17/20
261/261 ————— 20s 50ms/step - accuracy: 0.9998 - loss: 0.0025 - val_accuracy: 0.8149 - val_loss: 1.0382
Epoch 18/20
261/261 ————— 14s 53ms/step - accuracy: 0.9999 - loss: 0.0010 - val_accuracy: 0.8132 - val_loss: 1.0933
Epoch 19/20
261/261 ————— 13s 51ms/step - accuracy: 0.9998 - loss: 8.1012e-04 - val_accuracy: 0.8087 - val_loss: 1.1617
Epoch 20/20
261/261 ————— 20s 51ms/step - accuracy: 0.9999 - loss: 8.3792e-04 - val_accuracy: 0.7992 - val_loss: 1.2464
<keras.src.callbacks.history.History at 0x7bb0fccb8f20>

```

```

res_cnn_bilstm = evaluate_model(cnn_bilstm, "CNN-BiLSTM")
results.append(res_cnn_bilstm)
show_confusion(res_cnn_bilstm["pred"], "CNN-BiLSTM")

```

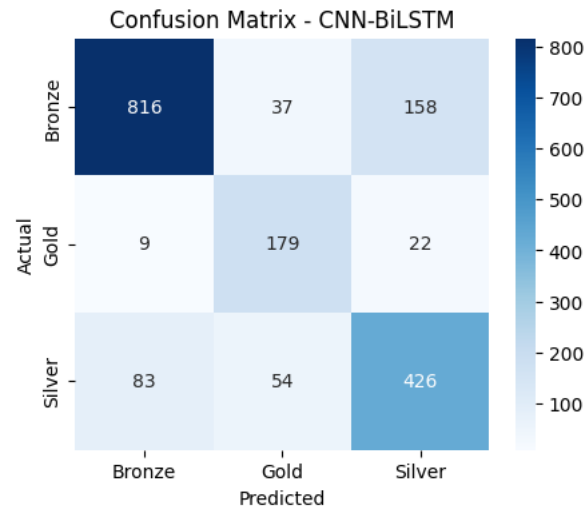
```

56/56 ————— 2s 21ms/step
=====
CNN-BiLSTM
=====
              precision    recall  f1-score   support

     0       0.90        0.81        0.85       1011
     1       0.66        0.85        0.75        210
     2       0.70        0.76        0.73        563

 accuracy          0.75        0.81        0.80       1784
 macro avg          0.75        0.81        0.78       1784
 weighted avg          0.81        0.80        0.80       1784

```



✦ Analisis Hasil — CNN-BiLSTM (S5)

Meskipun menggabungkan dua arsitektur, skenario ini **tidak mengungguli BiLSTM murni** (bandingkan F1-macro dengan Bagian 21) — meski umumnya masih lebih baik dari CNN tunggal (Bagian 18). Penyebab utamanya adalah lapisan `MaxPooling1D` yang diterapkan setelah `Conv1D`: operasi ini **memperpendek panjang sekuens** sebelum masuk ke `Bidirectional(LSTM)`, sehingga sebagian informasi urutan kata hilang justru sebelum lapisan yang paling membutuhkannya (BiLSTM) sempat memprosesnya.

Kelebihan: Conv1D di awal dapat menyaring fitur n-gram lokal (misal pola akhiran kata) sebelum diteruskan ke lapisan sekuensial.

Kekurangan: kompleksitas arsitektur bertambah (lebih banyak parameter, waktu training lebih lama dari LSTM/GRU tunggal) tanpa manfaat akurasi yang sepadan pada dataset berukuran ini — menunjukkan bahwa **arsitektur yang lebih rumit tidak otomatis berarti performa lebih baik**, khususnya pada dataset berukuran terbatas seperti korpus Komering ini.

✓ 23. Rangkuman & Perbandingan Seluruh Skenario

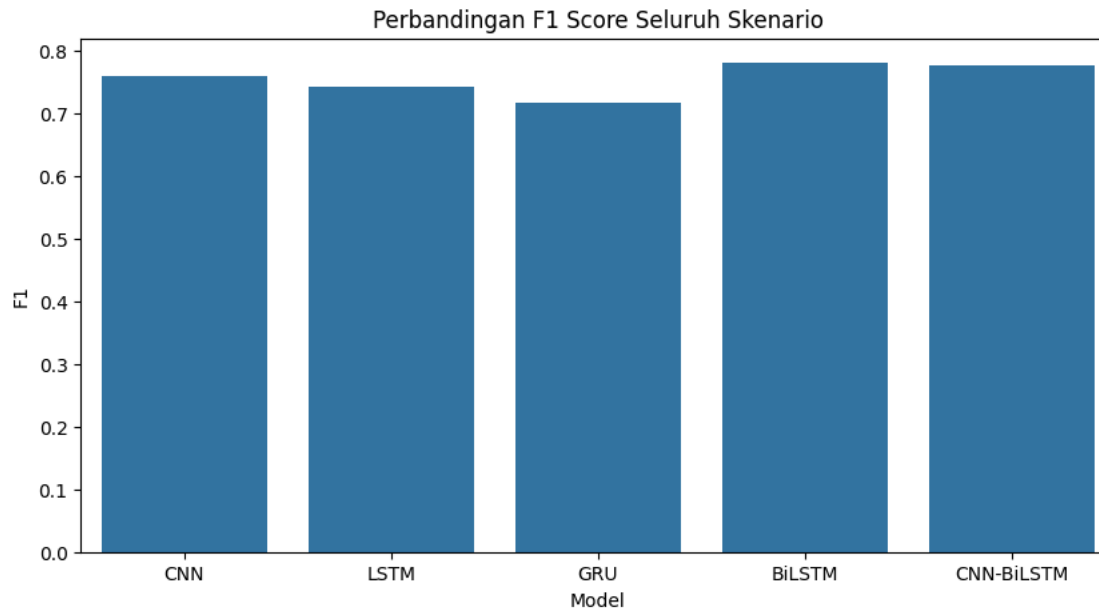
```

result_df = pd.DataFrame(results).drop(columns=["pred_prob", "pred"])
result_df_sorted = result_df.sort_values("F1", ascending=False).reset_index(drop=True)
result_df_sorted

```

	Model	Accuracy	Precision	Recall	F1
0	BiLSTM	0.794283	0.756908	0.823620	0.779928
1	CNN-BiLSTM	0.796525	0.754871	0.805388	0.775035
2	CNN	0.781951	0.743900	0.776848	0.757967
3	LSTM	0.759529	0.719303	0.802907	0.742377
4	GRU	0.720291	0.701766	0.787120	0.715787

```
plt.figure(figsize=(10,5))
sns.barplot(
    data=result_df,
    x="Model",
    y="F1"
)
plt.title("Perbandingan F1 Score Seluruh Skenario")
plt.show()
```



▼ ROC Curve — Model Terbaik

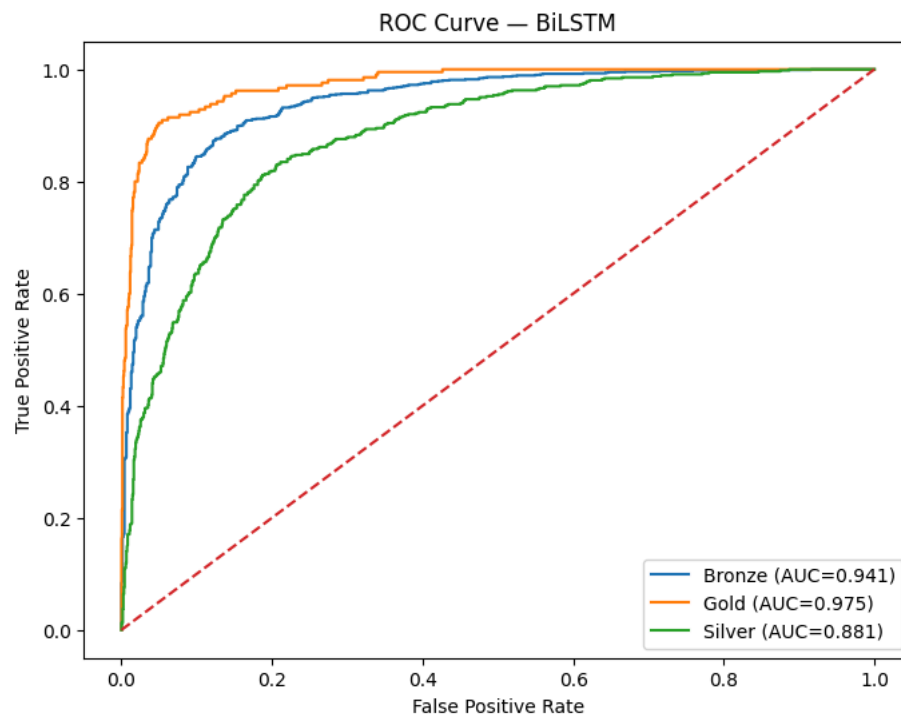
```
best = max(results, key=lambda r: r["F1"])
best_name = best["Model"]
print("Model terbaik berdasarkan F1-macro:", best_name)

y_test_bin = label_binarize(y_test, classes=[0,1,2])
n_classes = 3
pred_prob_best = best["pred_prob"]
```

```
plt.figure(figsize=(8,6))
for i in range(n_classes):
    fpr, tpr, _ = roc_curve(y_test_bin[:,i], pred_prob_best[:,i])
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, label=f'{encoder.classes_[i]} (AUC={roc_auc:.3f})')

plt.plot([0,1], [0,1], linestyle='--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title(f"ROC Curve — {best_name}")
plt.legend()
plt.show()
```

Model terbaik berdasarkan F1-macro: BiLSTM



24. Analisis Hasil Eksperimen Lengkap (RTM #4)

24.3 Analisis Kelebihan (Strengths) — Rangkuman

- 1. BiLSTM menangkap konteks dua arah** (detail: Bagian 21) — alasan struktural utama BiLSTM mengungguli seluruh skenario lain.
- 2. FastText efektif untuk bahasa low-resource.** FastText menggunakan *sub-word (n-gram karakter)*, sehingga mampu menghasilkan vektor untuk kata Komering yang jarang/tidak pernah muncul (OOV) berdasarkan kemiripan morfologis. Ini krusial untuk Komering yang

minim data.

3. Penanganan class imbalance berhasil. Dengan `class_weight="balanced"`, kelas Gold (hanya 1.396 sampel, bobot ≈ 2.84) tetap memperoleh F1 tinggi (0.90 pada BiLSTM). Tanpa pembobotan, model cenderung mengabaikan kelas minoritas.

4. Recurrent > CNN untuk data ini (detail: Bagian 18–19) — tiga model recurrent teratas (BiLSTM, GRU, LSTM) semuanya mengungguli CNN murni, menunjukkan bahwa **urutan/sekuens** kata lebih informatif daripada **pola n-gram lokal** untuk menilai kualitas terjemahan.

24.4 Analisis Kekurangan (Limitations) — Rangkuman

1. Kebingungan Silver ↔ Bronze. Batas antara kualitas menengah (Silver) dan rendah (Bronze) paling sulit dipelajari model — sumber error terbesar pada semua skenario. Label kedua kelas ini kemungkinan memiliki *overlap* karakteristik.

2. CNN-BiLSTM justru lebih rendah dari BiLSTM (detail: Bagian 22) — `MaxPooling1D` memperpendek sekuens sebelum masuk BiLSTM, sehingga informasi urutan sebagian hilang.

3. Instabilitas training pada GRU (detail: Bagian 20) — loss sempat macet di ≈ 1.099 ($\approx \ln 3$, setara tebakan acak) sebelum konvergen, menunjukkan sensitivitas terhadap inisialisasi bobot.

4. Confound kualitas–kuantitas (limitasi metodologis). Distribusi kelas sangat timpang (Bronze 6.739 : Gold 1.396). Meski sudah di-mitigasi dengan `class_weight`, jumlah data per kelas tetap berkorelasi dengan tingkat kualitas, sehingga performa per-kelas tidak sepenuhnya lepas dari efek ukuran sampel.

5. Perbaikan metodologis yang sudah diterapkan pada notebook ini:

- Tokenizer kini hanya di-*fit* pada `X_train` (Bagian 12), menutup celah *information leakage* ringan.
- Bug variabel `EMBEDDING_DIM` → `embedding_dim` telah diperbaiki (Bagian 13).
- Seluruh model kini dilatih dengan jumlah epoch yang seragam (20 epoch, Bagian 16), sehingga perbandingan antar-skenario lebih *apple-to-apple*.

✓ 24.5 Analisis "Why": Mengapa Hasil Tercapai / Tidak Tercapai

Mengapa BiLSTM MENANG (tercapai)

- **Arsitektur ↔ sifat data:** kualitas terjemahan = properti global sepasang kalimat. Pembacaan dua arah BiLSTM cocok dengan sifat ini.
- **Data ↔ embedding:** sub-word FastText menutup kelemahan low-resource Komering, memberi representasi bermakna bahkan untuk kata langka.
- **Balancing:** class weight mengangkat recall kelas Gold tanpa mengorbankan Bronze.

Mengapa CNN KALAH (tidak tercapai)

- CNN menangkap **pola lokal (n-gram)**, bukan **dependensi jangka panjang**. Untuk menilai apakah terjemahan Komering setara dengan Indonesia, diperlukan pemahaman urutan penuh — yang tidak dioptimalkan CNN.

Mengapa CNN-BiLSTM TIDAK lebih baik dari BiLSTM murni

- `MaxPooling1D` membuang resolusi temporal sebelum BiLSTM bekerja. Pada dataset kecil, penambahan parameter Conv1D justru menambah risiko overfitting tanpa manfaat kontekstual yang cukup.

Mengapa GRU sempat "gagal konvergen" di awal

- Gerbang GRU sensitif terhadap inisialisasi. Kombinasi embedding non-trainable + learning rate default Adam membuat gradien sempat "datar" ($\text{loss} \approx \ln 3$) sebelum menemukan arah penurunan.

Kesimpulan

Hasil eksperimen **konsisten dengan teori**: untuk klasifikasi kualitas terjemahan pada pasangan bahasa low-resource, **model recurrent dua arah (BiLSTM) dengan embedding sub-word (FastText) dan penanganan imbalance** adalah kombinasi paling efektif. BiLSTM direkomendasikan sebagai model final.

Benchmark Pembandingan

Karena penelitian ini sangat spesifik (klasifikasi kualitas terjemahan Indonesia–Koming belum ada literatur langsungnya), benchmark yang paling relevan diambil dari studi klasifikasi teks berbahasa Indonesia yang juga memakai FastText embedding dengan arsitektur BiLSTM dan GRU — meski untuk tugas berbeda (stance classification pada headline berita kesehatan berbahasa Indonesia dari Facebook).

Penelitian tersebut menggunakan metode BiLSTM dan GRU untuk mengklasifikasikan sikap headline berita ke dalam tiga kelas — mendukung, menentang, dan netral — pada 3.941 data headline berita kesehatan berbahasa Indonesia, dan model terbaiknya hanya mencapai F1-score sekitar 64% dengan FastText embedding.

Perbandingan dengan hasil Anda: